Engineer to Excel

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1410 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I CH.V.Ramayogi/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:CH.V.Ramayogi

Annexure A

SCENARIO-BASED TASK

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

ANSWER:

1. Introduction

A syntax analyzer, also called a parser, is an important phase of a compiler. It receives a sequence of tokens from the lexical analyzer and checks whether the sequence follows the production rules of a context-free grammar (CFG). If the input satisfies the grammar, the parser constructs a parse tree or an abstract syntax tree (AST). Otherwise, it reports a syntax error.

An LL(1) parser is a top-down predictive parser. The first L indicates that input is scanned from left to right, the second L indicates that a leftmost derivation is constructed, and 1 indicates that one lookahead token is used to select the next production.

An LL(1) parser uses a parsing table to decide which production should be applied. The table is constructed using the FIRST and FOLLOW sets of grammar symbols. If multiple productions are entered into the same cell, the parser cannot uniquely determine which production to apply.

2. Role of the FIRST Set

The FIRST set of a grammar symbol contains all terminals that can appear at the beginning of strings derived from that symbol. If the symbol can derive the empty string, epsilon (ε) is included.

For a production A → α, the FIRST set of α helps determine which input symbols can select that production.

For example, consider:

A → aB | bC

The corresponding FIRST sets are:

FIRST(aB) = {a}

FIRST(bC) = {b}

If the next input token is a, the parser selects A → aB. If the next token is b, it selects A → bC. Since the FIRST sets are disjoint, the parser can make a unique decision.

3. Role of the FOLLOW Set

The FOLLOW set of a non-terminal A contains the terminals that may immediately follow A in a valid sentential form. The end marker $ is included in FOLLOW of the start symbol.

FOLLOW is especially important when a production can derive ε.

For a production A → ε, the parser enters this production into M[A, b] for every terminal b in FOLLOW(A). If $ belongs to FOLLOW(A), the production is also entered into M[A, $].

Therefore, FIRST determines which production to select when the production begins with an input terminal, while FOLLOW helps determine when an empty production should be selected.

4. Construction of an LL(1) Parsing Table

The parsing table is represented by M[A, a], where A is a non-terminal and a is an input terminal or the end marker $.

The standard construction rules are:

1. For each production A → α, place the production in M[A, a] for every terminal a in FIRST(α), excluding ε.
2. If ε belongs to FIRST(α), place A → α in M[A, b] for every b in FOLLOW(A).
3. If any parsing-table cell receives more than one production, a conflict exists.

For an LL(1) grammar, every applicable parsing-table cell must contain at most one production.

5. Identification of the Key Issue

Consider the following grammar:

A → aB | aC

B → b

C → c

The FIRST sets of the two alternatives of A are:

FIRST(aB) = {a}

FIRST(aC) = {a}

Both alternatives begin with the same terminal a. Therefore, the parsing table contains two productions in the same cell:

Non-terminal	Input a
A	A → aB
A	A → aC

This is an LL(1) conflict because the parser cannot determine which alternative to select by examining only the next input token.

A conflict may also arise when one alternative derives ε and another alternative can begin with a terminal that belongs to FOLLOW(A). In that case, the FIRST/FOLLOW conditions overlap.

6. Effect on Deterministic Parsing

Deterministic parsing means that the parser can select exactly one action at each step using the current parser state and available lookahead.

When an LL(1) parsing-table cell contains multiple productions, the parser has no unique production-selection rule. This can lead to several problems:

- The parser may select an incorrect production.
- The parser may construct an incorrect derivation or parse tree.
- The parser may reject a valid input if the wrong alternative is selected.
- The implementation may require backtracking or additional lookahead, which no longer follows the standard LL(1) predictive parsing method.

The presence of a conflict indicates that the grammar, as written, is not LL(1). It does not necessarily mean that the language itself cannot be parsed using an LL(1) grammar.

7. Grammar Modification Using Left Factoring

The appropriate solution for the example is left factoring. Left factoring extracts the common prefix and postpones the choice until enough input has been examined.

Original grammar:

$A \rightarrow aB \mid aC$

$B \rightarrow b$

$C \rightarrow c$

After left factoring:

$A \rightarrow aA'$

$A' \rightarrow B \mid C$

$B \rightarrow b$

$C \rightarrow c$

Now, after reading a, the parser examines the next token to choose between B and C.

The relevant FIRST sets are:

FIRST(B) = {b}

FIRST(C) = {c}

The alternatives of A′ have disjoint FIRST sets, so the parser can distinguish them using one lookahead token.

For this example, the transformed grammar is suitable for LL(1) parsing, provided the complete grammar has no other conflicts.

If the conflict is caused by left recursion, left recursion must also be eliminated. After every transformation, the FIRST and FOLLOW sets and the parsing table must be recalculated.

8. Decision and Justification

The student should not resolve the conflict by arbitrarily choosing one production. Doing so may cause valid strings to be rejected or incorrectly parsed.

The correct decision is to:

1. Identify the conflicting productions.
2. Calculate their FIRST sets and check the relevant FOLLOW sets.
3. Apply left factoring if alternatives share a common prefix.
4. Eliminate left recursion if it exists.
5. Recalculate the FIRST and FOLLOW sets.
6. Reconstruct the parsing table and verify that no cell contains multiple productions.

Conclusion

FIRST and FOLLOW sets are essential for constructing an LL(1) parsing table and selecting productions deterministically. Multiple entries in a single cell indicate that the grammar does not satisfy the LL(1) conditions. Left factoring, left-recursion elimination, and systematic FIRST/FOLLOW analysis can remove such conflicts when an appropriate LL(1) form exists. Therefore, the grammar must be transformed and validated before implementing the predictive parser.

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

ANSWER:

1. Introduction

A syntax analyzer, also called a parser, is an important phase of a compiler. It receives tokens from the lexical analyzer and checks whether they follow the production rules of a context-free grammar (CFG). If the token sequence is valid, the syntax analyzer constructs a parse tree or an abstract syntax tree (AST).

A parse tree represents the grammatical structure of an input expression, whereas an AST represents its essential operators and operands without unnecessary grammar symbols.

For arithmetic expressions, the structure of the tree determines the order in which operations are performed. Therefore, the syntax analyzer must follow the correct operator precedence and associativity rules.

Consider the expression:

id + id - id

In conventional arithmetic, addition and subtraction have the same precedence and are left-associative. Therefore, the correct grouping is:

[

(id_1 + id_2) - id_3

]

The syntax analyzer must construct a tree that represents this grouping to ensure correct evaluation.

2. Understanding Operator Associativity

Operator associativity determines how operators with the same precedence are grouped when they appear consecutively in an expression.

There are two common types of associativity.

A. Left Associativity

In left associativity, operators are grouped from left to right.

For example:

a - b - c

The correct grouping is:

[

(a-b)-c

]

The first subtraction is performed before the second subtraction.

Similarly, the expression:

a + b - c

is grouped as:

[

(a+b)-c

]

Addition and subtraction are conventionally left-associative.

B. Right Associativity

In right associativity, operators are grouped from right to left.

For example, if exponentiation is defined as right-associative:

a ^ b ^ c

The grouping is:

[

a^{(b^c)}

]

The right-hand exponentiation is grouped first.

Thus, associativity determines the structure of an expression when multiple operators of equal precedence appear together.

3. Analysis of the Given Expression

The expression supplied in the scenario is:

id + id - id

Let the identifiers represent three operands: (id_1), (id_2), and (id_3).

Since addition and subtraction have equal precedence and are left-associative, the expression should be interpreted as:

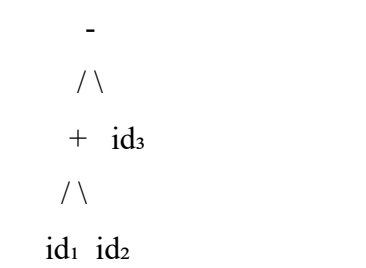
[

(id_1+id_2)-id_3

]

The syntax analyzer must recognize the addition operation as the left operand of the final subtraction operation.

The correct AST is shown below:



In this tree:

- The root node is subtraction (-).
- The left child represents addition (+).
- The right child is the third identifier (id_3).
- The addition node has id_1 and id_2 as its children.

Therefore, the tree represents the expression ((id_1+id_2)-id_3).

4. Identification of the Key Issue

The key issue in this scenario is incorrect operator associativity during parse tree or AST construction.

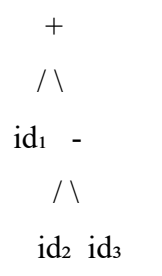
If the grammar or parser implementation does not enforce left associativity, the syntax analyzer might generate the following alternative grouping:

[

id_1+(id_2-id_3)

]

Its AST would be:



This tree represents a different expression structure.

The problem occurs because the parser has grouped the subtraction operation inside the right operand of addition rather than grouping the addition operation first.

Different groupings can produce different numerical results, particularly when subtraction is involved.

For example, consider:

10 - 5 - 2

Correct left-associative evaluation:

[

(10-5)-2=5-2=3

]

Incorrect right-associated evaluation:

[

10-(5-2)=10-3=7

]

The results are different: the correct result is 3, while the incorrectly grouped expression produces 7.

This demonstrates that the syntax analyzer must construct the correct tree rather than simply recognizing the individual tokens.

5. How Grammar Rules Influence Parse Tree Construction

A context-free grammar defines the valid structures that a parser can recognize. The arrangement of production rules influences the structure of the resulting parse tree.

Consider the following grammar:

E → E + T

| E - T

| T

T → id

Here:

- E represents an expression.
- T represents a term.
- id represents an identifier.

The productions E → E + T and E → E - T describe expressions formed by combining an existing expression with another term.

The left-recursive structure supports left-associative grouping when the parser constructs the AST according to these productions.

For the expression id + id - id, the grammar allows the following derivation:

E

⇒ E - T

⇒ E + T - T

⇒ T + T - T

⇒ id + id - id

The derivation corresponds to the left-associated structure:

[

(id_1+id_2)-id_3

]

The final AST must retain this grouping.

However, a grammar transformation or a particular parser implementation must preserve the intended semantics. Merely accepting the expression is not sufficient; the resulting tree must also represent the correct operations.

6. Removing Left Recursion for Predictive Parsing

The preceding grammar contains left recursion. A conventional recursive-descent predictive parser cannot use this grammar directly because repeatedly expanding E → E + T or E → E - T can cause infinite recursion.

Therefore, the grammar can be transformed as follows:

E → T E′

E′ → + T E′

| - T E′

| ε

$T \rightarrow id$

Here, E′ represents the continuation of an expression after its first term.

This grammar is suitable for expressing repeated addition and subtraction without direct left recursion. However, the transformed right-recursive structure does not automatically guarantee a left-associated AST.

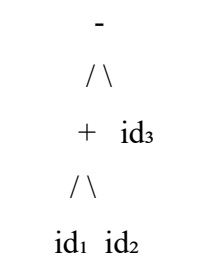
The parser must construct the AST using a left-to-right accumulation strategy.

AST Construction Procedure

For the expression $id_1 + id_2 - id_3$, the parser performs these steps:

1. Recognize id_1 and initialize the current expression with this identifier.
2. Read the + operator and recognize id_2.
3. Create an addition node with the current expression as its left child and id_2 as its right child.
4. Read the - operator and recognize id_3.
5. Create a subtraction node with the current addition expression as its left child and id_3 as its right child.
6. Return the final AST.

The final structure is:



This process preserves left associativity even though the grammar has been transformed to remove left recursion.

7. Difference Between Parse Tree and Abstract Syntax Tree

A parse tree contains all grammar symbols used during derivation, including non-terminals such as E and T. An AST removes unnecessary grammar details and emphasizes the operators and operands.

For example, the parse tree for an expression may contain multiple levels of E and T nodes. The AST simplifies these levels into a structure directly representing the intended computation.

In the given scenario, the AST should clearly show that addition is the left operand of subtraction.

This distinction is important because the compiler may use the AST during semantic analysis, intermediate-code generation, and later compilation stages. An incorrect AST can lead to incorrect generated instructions and program results.

8. Decision and Justification

To ensure correct associativity, the syntax analyzer must use grammar rules and AST-construction logic that preserve the language's operator semantics.

The following decisions should be taken:

1. Define associativity clearly: Specify that addition and subtraction are left-associative.
2. Use appropriate grammar productions: Use left-recursive expression productions for a compatible bottom-up parser, or remove left recursion for predictive parsing.
3. Preserve left-to-right grouping: When using a transformed grammar, build the AST using a left fold or accumulator.
4. Maintain precedence rules: Ensure that operators with different precedence levels are represented at the appropriate grammar levels.
5. Validate the AST: Test expressions containing several operators to verify that the tree matches the intended grouping.
6. Test numerical results: Use examples such as 10 - 5 - 2 to confirm that left associativity is correctly implemented.

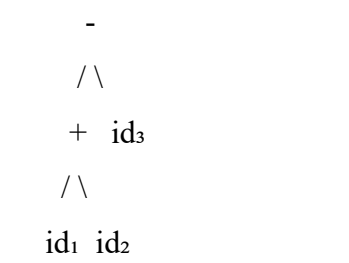
The syntax analyzer should not choose a grouping arbitrarily. It must construct the tree according to the rules defined by the programming language.

9. Expected Result

For the input expression:

id + id - id

The expected AST is:



The expression represented by this tree is:

[

(id_1+id_2)-id_3

]

This tree follows the conventional left-associative behavior of addition and subtraction.

10. Conclusion

The incorrect parse tree generation scenario demonstrates the importance of operator associativity in compiler design. A syntax analyzer must do more than verify that an expression follows the grammar; it must also construct a parse tree or AST that represents the intended computation.

For the expression id + id - id, addition and subtraction have equal precedence and are left-associative. Therefore, the correct grouping is ((id_1+id_2)-id_3). If the parser incorrectly groups the expression as (id_1+(id_2-id_3)), the evaluation may produce an incorrect result.

The appropriate decision is to define associativity explicitly, use a suitable grammar, and implement correct AST-construction logic. For predictive parsing, left recursion must be eliminated, and left-to-right accumulation should be used to preserve the intended grouping. These measures ensure correct parse tree generation, accurate expression evaluation, and reliable compiler behavior**.**

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

ANSWER:

1. Introduction

A syntax analyzer is an important phase of a compiler that checks whether the sequence of tokens generated by the lexical analyzer follows the grammatical rules of a programming language. It uses a context-free grammar (CFG) to determine whether the input is syntactically correct.

The lexical analyzer identifies individual tokens such as identifiers, keywords, operators, and constants. However, even if every token is valid individually, the entire sequence may be invalid because the tokens are arranged incorrectly.

For example, consider the expression:

a + * b

The tokens a, +, *, and b are individually valid tokens. However, the expression is syntactically incorrect because an operand is missing between the addition and multiplication operators.

The syntax analyzer detects this problem by checking whether the token sequence can be generated by the grammar. If the sequence violates the grammar, the parser reports a syntax error instead of generating a valid parse tree.

2. Context-Free Grammar for Arithmetic Expressions

Consider the following grammar for arithmetic expressions:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id \mid (E)$

Where:

- E represents an expression.
- T represents a term.
- F represents a factor.
- id represents an identifier.
- + represents the addition operator.
- * represents the multiplication operator.
- (E) represents an expression enclosed in parentheses.

This grammar defines how identifiers, operators, and parenthesized expressions can be combined to form valid arithmetic expressions.

Examples of valid expressions include:

a + b

a * b

a + b * c

(a + b) * c

However, the expression a + * b is invalid because the grammar requires a valid term after the addition operator.

For a predictive parser, the grammar must first be transformed to eliminate left recursion. The original grammar is useful for describing the expression structure, but it cannot be used directly by a conventional recursive-descent predictive parser.

3. Identification of the Key Issue

The key issue in this scenario is a **syntax error caused by an invalid arrangement of operators and a missing operand**.

Consider the token sequence:

id + * id

The parser recognizes the first identifier as a valid factor, term, and expression. It then encounters the addition operator.

After the + operator, the grammar requires a valid term containing an identifier or another valid factor. Instead, the next token is *.

The multiplication operator requires an operand on its left and an operand on its right. In this expression, there is no operand between + and *.

Therefore, the input does not satisfy the grammar's production rules.

The error is syntactic rather than necessarily lexical because each token can be recognized individually, but the sequence does not form a valid expression.

4. How the Syntax Analyzer Detects the Error

The syntax analyzer compares the input tokens against the grammar rules and determines whether a valid derivation exists.

For the expression:

a + * b

the parsing process can be explained as follows:

1. The lexical analyzer identifies a as an identifier.
2. The syntax analyzer recognizes a as a valid expression component.
3. The parser reads the + operator and expects another valid term.
4. The next token is *, which cannot begin a factor under the given grammar.
5. The parser finds that no valid production can match the expected grammar symbol and lookahead token.
6. The syntax analyzer reports a syntax error and may initiate an error-recovery procedure.

A suitable diagnostic message would be:

Syntax Error:

Unexpected '*' after '+'.

Expected an identifier or '('.

An operand is missing.

This message identifies the unexpected token and explains the likely cause of the error.

5. Error Detection Using Predictive Parsing

Predictive parsing is a top-down parsing technique that selects productions using the current non-terminal and a lookahead token. An LL(1) predictive parser uses one lookahead token and a parsing table constructed from FIRST and FOLLOW sets.

To use a conventional predictive parser, the grammar must be transformed to remove left recursion.

The transformed grammar is:

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow id \mid (E)$

In this grammar, E′ represents the continuation of an expression, while T′ represents the continuation of a term.

Step-by-Step Error Detection

Consider the input:

id + * id $

The end marker $ indicates the end of the input.

The parser processes the input as follows:

Step 1: The parser starts with the start symbol E.

Step 2: It expands $E \rightarrow T E'$ and recognizes the first identifier through $F \rightarrow id$.

Step 3: The parser recognizes the identifier as a term and expression component.

Step 4: It encounters + and uses the expression-continuation production $E' \rightarrow + T E'$.

Step 5: After matching +, the parser expects a term. A term must begin with a valid factor, such as an identifier or an opening parenthesis.

Step 6: The next input token is *. Since * cannot begin a factor, the parser cannot select a valid production for the expected factor.

Step 7: The parser reports a syntax error.

The precise stack operations depend on the parser implementation, but the essential reason for rejection remains the same: the token * appears where an operand is required.

6. Error Detection Using Shift-Reduce Parsing

Shift-reduce parsing is a bottom-up parsing technique that constructs a parse tree from the input tokens. It uses a stack and performs shift and reduce operations.

Shift Operation

The parser moves the next input token onto the stack.

Reduce Operation

The parser replaces a sequence of symbols matching the right-hand side of a grammar production with the corresponding non-terminal.

For example, the production:

$F \rightarrow id$

allows an identifier to be reduced to a factor.

Processing the Invalid Expression

For the input:

id + * id

the parser recognizes the first identifier and may reduce it to a factor, term, and expression component according to its parsing table.

After processing the addition operator, the parser encounters * without a valid right operand for the preceding addition.

At this point, the parser cannot perform a valid reduction that completes the required expression. Depending on its state and parsing table, it may find that no valid shift or reduce action is available.

The parser then reports a syntax error.

Unlike a predictive parser, a shift-reduce parser uses its stack and state information to determine whether the next action is valid.

7. Error Recovery Techniques

Error detection identifies an invalid expression, while error recovery attempts to continue parsing the remaining input. A compiler should provide meaningful diagnostics and avoid producing excessive error messages for a single mistake.

The following recovery techniques are commonly used.

A. Panic-Mode Recovery

Panic-mode recovery is a simple and widely used error-recovery technique. When a syntax error occurs, the parser skips input tokens until it encounters a synchronization token.

Common synchronization tokens include:

- Semicolon ;
- Closing parenthesis)
- Closing brace }
- End of input $

For example:

a + * b;

c = d + e;

If the first expression contains a syntax error, the parser may skip to the semicolon and then continue parsing the next statement.

Advantage: It is easy to implement and helps prevent repeated errors caused by the same invalid expression.

Limitation: Some valid input tokens may be skipped, so the parser may lose part of the source code during recovery.

B. Phrase-Level Recovery

Phrase-level recovery makes a small local correction to continue parsing. The parser may insert a missing token, delete an unexpected token, or replace a token with an expected one.

For the expression:

a + * b

one possible recovery is to delete the unexpected * token and continue with b. The parser can then recover an expression resembling:

a + b

This is only a recovery action; it does not mean the original expression was valid.

Advantage: It can preserve more of the input and allow parsing to continue from a nearby location.

Limitation: Incorrect recovery decisions may lead to misleading diagnostics or an incorrect recovered syntax tree.

C. Error Productions

Error productions are additional grammar rules designed to recognize common programming mistakes. For example, a parser may include special handling for two consecutive binary operators. When the pattern + * occurs in an expression, the parser can produce a specific diagnostic message indicating that an operand is missing.

Advantage: Error messages can be more meaningful and specific.

Limitation: Error productions must be designed carefully so that they do not introduce ambiguity or unintentionally accept invalid expressions.

D. Global Correction

Global correction attempts to find the smallest sequence of insertions, deletions, or replacements that would transform an invalid input into a valid one.

Although this approach can produce a theoretically useful correction, it is computationally expensive and is not commonly used as the primary recovery method in ordinary compilers.

For practical compiler implementations, panic-mode and phrase-level recovery are generally more straightforward.

8. Importance of Clear Error Messages

A compiler should explain not only that an error occurred but also where it occurred and what the parser expected.

For the expression:

a + * b

an effective error message is:

Syntax Error at '*':

Unexpected multiplication operator after '+'.

Expected an identifier, constant, or '('.

Possible cause: Missing operand.

A useful diagnostic should include:

1. The location of the unexpected token.
2. The token that caused the error.
3. The expected token or grammar category.
4. A likely explanation of the problem.
5. A recovery action, where appropriate.

Clear error messages help programmers identify and correct syntax errors more efficiently.

9. Decision and Justification

The correct decision is to retain the grammar's syntactic rules while improving error detection and recovery.

The compiler should not accept a + * b as a valid expression merely to avoid reporting an error. Instead, it should identify the missing operand and use an appropriate recovery method if further parsing is possible.

The following measures should be implemented:

1. **Use a well-defined grammar:** Ensure that operators appear only in valid grammatical positions.
2. **Validate parser actions:** Detect when no valid production or shift-reduce action is available.
3. **Provide meaningful diagnostics:** Report the unexpected token and expected grammar category.
4. **Apply suitable recovery:** Use panic-mode recovery, phrase-level recovery, or carefully designed error productions.
5. **Avoid cascading errors:** Resume parsing from a suitable synchronization point to prevent one mistake from producing many misleading messages.
6. **Preserve correctness:** Clearly distinguish successful parsing from error recovery so that invalid input is never silently treated as valid.

These decisions improve the compiler's usability while maintaining the correctness of syntax analysis.

10. Expected Result

For the given input:

a + * b

the expected parser result is:

Status: Syntax Error

Reason: The multiplication operator appears where an operand is required after the addition operator.

Expected input: An identifier, constant, or opening parenthesis.

Suggested recovery: Skip the unexpected token or synchronize at a suitable delimiter, depending on the surrounding program structure.

The compiler should then continue parsing only when a valid parser state has been restored.

11. Conclusion

The invalid expression handling scenario demonstrates that valid individual tokens do not necessarily form a valid expression. In the input a + * b, the syntax analyzer detects a syntax error because the multiplication operator appears without a valid operand between the addition and multiplication operators.

A predictive parser detects the error when no valid production can be selected from the parsing table. A shift-reduce parser detects it when no valid parsing action is available for the current state and lookahead token.

Panic-mode recovery, phrase-level recovery, and carefully designed error productions can improve the compiler's ability to continue parsing after an error. Therefore, the appropriate decision is to combine a well-defined grammar with accurate error detection, clear diagnostic messages, and suitable recovery strategies. This approach ensures reliable syntax analysis without compromising the correctness of the compiler.

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar S → if E then S | if E then S else S | a. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

ANSWER:

1. Introduction

A syntax analyzer, also known as a parser, is a major phase of a compiler that checks whether a sequence of tokens follows the production rules of a context-free grammar (CFG). It constructs a parse tree or an abstract syntax tree (AST) representing the syntactic structure of the source program.

Conditional statements are important programming constructs that allow a program to execute different statements depending on whether a condition is true or false. The if-then statement executes a statement when a condition is satisfied, while the if-then-else statement provides an alternative when the condition is not satisfied.

Consider the grammar provided in the scenario:

S → if E then S

| if E then S else S

| a

Here:

- S represents a statement.
- E represents a Boolean condition or expression.
- if is the conditional keyword.
- then separates the condition from the statement.
- else introduces an alternative statement.
- a represents a simple statement.

The grammar allows both conditional statements with an else clause and conditional statements without an else clause. However, it does not specify how an else should be associated when conditional statements are nested. This creates the dangling else problem.

2. Understanding Grammar Ambiguity

A context-free grammar is ambiguous if at least one valid input string has more than one distinct parse tree or more than one distinct leftmost derivation.

An ambiguous grammar does not uniquely determine the syntactic structure of every valid string. Consequently, a parser may encounter conflicts when attempting to choose a production or construct a parse tree.

In the given grammar, the two conditional productions share a common prefix:

S → if E then S

S → if E then S else S

Both productions begin with if E then S. The second production additionally contains else S.

When the parser encounters a nested conditional statement, it may not immediately know whether the shorter production should be completed or whether the longer production should be used to associate an else clause with a conditional statement.

This creates ambiguity in the grammatical structure.

3. Understanding the Dangling Else Problem

The dangling else problem occurs when a nested conditional statement contains an else clause that can be associated with either an inner if statement or an outer if statement.

Consider the following input:

if E1 then if E2 then a else a

This statement contains two if keywords but only one else keyword. The original grammar allows two possible interpretations.

Interpretation 1: Else Belongs to the Inner If

if E1 then

(if E2 then a else a)

In this interpretation, the outer if checks condition E1. If E1 is true, the inner if checks E2. The else belongs to the inner conditional statement.

The structure can be represented by the following tree:

```
        if E1
       /    \
    then   if E2
           /   \
        then  else
        |     |
        a     a
```

This tree represents a nested conditional in which the inner if has both a then and an else branch.

Interpretation 2: Else Belongs to the Outer If

if E1 then

(if E2 then a)

else

a

In this interpretation, the inner if has no else clause. The else belongs to the outer conditional statement.

Its simplified structure is:

```
          if E1
         /    \
       then    else
        |       |
      if E2     a
      /   \
    then   a
```

Both interpretations use the same sequence of input tokens. However, the conditional statements have different structures and meanings.

Therefore, the grammar permits multiple parse trees for the same input and is ambiguous.

4. Identification of the Key Issue

The main issue in this scenario is **the dangling else problem caused by an ambiguous conditional grammar**.

The ambiguity occurs because the grammar does not explicitly determine whether an else should be associated with the nearest unmatched if or with an outer conditional statement.

This creates several difficulties:

1. **Multiple parse trees:** The same source statement can have more than one syntactic interpretation.
2. **Parsing-table conflicts:** A parser may encounter multiple possible production choices.
3. **Shift-reduce conflicts:** A bottom-up parser may have to decide whether to shift an else token or reduce an if-then statement.
4. **Uncertain control flow:** Different associations of the else clause produce different program behavior.
5. **Difficult compiler implementation:** The grammar or parsing strategy must be modified to establish a consistent interpretation.

The parser should not arbitrarily choose an interpretation unless the programming language explicitly defines a rule for resolving the ambiguity.

5. Impact on Top-Down Parsing

A top-down parser begins with the start symbol and attempts to derive the input string using grammar productions.

In the given grammar, the two alternatives for S begin with the same sequence:

S → if E then S

S → if E then S else S

Because the productions share a common prefix, the parser may not have enough information to choose between them when it first encounters if.

A conventional LL(1) predictive parser uses only one lookahead token to select a production. The shared prefix prevents straightforward selection based on the initial if token alone.

Left factoring can remove the common prefix, but it does not by itself eliminate the fundamental dangling else ambiguity. A separate rule or an unambiguous grammar is still required.

6. Impact on Bottom-Up Parsing

A bottom-up parser constructs a parse tree by shifting input tokens onto a stack and reducing recognized sequences according to grammar productions.

When a nested conditional statement is followed by else, the parser may face two possible actions:

- **Reduce:** Complete the shorter production S → if E then S.
- **Shift:** Consume the else token so that the longer production S → if E then S else S can be used.

This is a shift-reduce conflict.

If the parser reduces the shorter production too early, the else may become associated with an outer conditional statement. If it shifts the else, the parser can associate it with the nearest unmatched if.

The correct action depends on the convention defined by the programming language.

7. Solution Using the Nearest-Unmatched-If Rule

A common solution is to define that an else belongs to the nearest preceding unmatched if.

Under this convention, the statement:

if E1 then if E2 then a else a

is interpreted as:

if E1 then

(if E2 then a else a)

The inner conditional receives the else clause because it is the nearest unmatched if.

This convention is used by many programming languages and provides a predictable interpretation of nested conditional statements.

For a bottom-up parser, the relevant shift-reduce conflict can be resolved by shifting else rather than reducing the shorter if-then production, when the language adopts the nearest-unmatched-if rule.

However, this conflict resolution is a parsing convention; it does not make the original grammar mathematically unambiguous.

8. Grammar Modification Using Matched and Unmatched Statements

A more formal solution is to rewrite the grammar by distinguishing matched and unmatched conditional statements.

The modified grammar is:

S → M | U

M → if E then M else M

| a

U → if E then S

| if E then M else U

Here:

- S represents any statement.
- M represents a matched statement, where every conditional statement has a corresponding else.
- U represents an unmatched statement, where a conditional statement still has an unmatched if.

Explanation of the Modified Grammar

Production 1: S → M | U

A statement may be either matched or unmatched.

Production 2: M → if E then M else M

This production represents a conditional statement with both then and else branches. The nested statements are also matched.

Production 3: M → a

A simple statement is considered matched because it contains no unmatched conditional.

Production 4: U → if E then S

This production represents an if-then statement without an else clause.

Production 5: U → if E then M else U

This production represents a conditional statement in which the else is associated with the nearest unmatched if within the nested statement.

The matched/unmatched distinction prevents an else from being associated arbitrarily with a more distant if.

The complete grammar should still be checked for its suitability for the chosen parsing algorithm. Eliminating ambiguity does not automatically guarantee that a grammar is LL(1).

9. Alternative Solution Using Explicit Delimiters

Another practical approach is to use explicit block delimiters, such as braces, to identify the statements belonging to each conditional.

For example:

```
if (E1) {
   if (E2) {
      a;
   } else {
      a;
   }
}
```

The braces clearly show that the else belongs to the inner conditional statement.

This approach provides several benefits:

- It makes nested conditions easier to understand.
- It reduces ambiguity in the source code.
- It improves the readability of complex conditional statements.
- It helps the parser identify the boundaries of statement blocks.

Programming languages may use braces, indentation rules, explicit terminators, or other syntactic conventions to clarify conditional structures.

10. Decision and Justification

The syntax analyzer should not attempt to resolve the dangling else problem through arbitrary production selection. The programming language must define a clear association rule, and the grammar or parser implementation must enforce that rule.

The recommended decisions are:

1. **Adopt a clear association rule:** Associate each else with the nearest unmatched if.
2. **Modify the grammar:** Use matched and unmatched statement productions to represent the intended structures.
3. **Resolve parser conflicts appropriately:** For a bottom-up parser using the nearest-unmatched-if convention, shift the else when required by the parser's conflict-resolution rules.
4. **Use explicit delimiters where appropriate:** Braces or other block delimiters can make the intended association clear.
5. **Validate the grammar:** Check the revised grammar for ambiguity and determine whether it satisfies the requirements of the chosen parser.
6. **Test nested conditional statements:** Verify cases containing multiple if statements and different numbers of else clauses.

These decisions ensure that the syntax analyzer constructs the intended parse tree consistently.

11. Expected Result

For the input:

if E1 then if E2 then a else a

the parser should follow the nearest-unmatched-if convention and construct the structure:

if E1 then

(if E2 then a else a)

The inner if has both a then and an else branch, while the outer if does not have an else clause.

This gives the statement a single intended interpretation under the chosen convention.

12. Conclusion

The ambiguity in conditional statements arises because the grammar permits an else clause to be associated with either an inner or an outer if statement. This is known as the dangling else problem.

The ambiguity can lead to multiple parse trees, shift-reduce conflicts, and uncertainty in the interpretation of program control flow. The syntax analyzer must therefore follow a clearly defined association rule.

The recommended solution is to associate each else with the nearest unmatched if and enforce this rule using a matched/unmatched grammar, an appropriate parsing strategy, or explicit block delimiters. The revised grammar must be validated for the selected parsing technique.

5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar E → E + E | E * E | id. While parsing the input id + id * id, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

ANSWER:

1. Introduction

A syntax analyzer, also known as a parser, is an important phase of a compiler. It receives tokens from the lexical analyzer and checks whether they follow the production rules of a context-free grammar (CFG). If the expression is syntactically valid, the parser constructs a parse tree or an abstract syntax tree (AST).

In arithmetic expressions, the syntax analyzer must correctly interpret operator precedence and associativity. Operator precedence determines which operation should be grouped first, while associativity determines how operators with equal precedence are grouped.

Consider the grammar given in the scenario:

E → E + E

| E * E

| id

This grammar supports addition, multiplication, and identifiers. However, it does not explicitly define the precedence of addition and multiplication or the associativity of these operators.

For the input expression:

id + id * id

the intended interpretation under conventional arithmetic rules is:

[

id_1+(id_2*id_3)

]

Multiplication has higher precedence than addition. Therefore, the multiplication operation must be grouped before the addition operation.

2. Understanding Operator Precedence

Operator precedence determines which operator binds more strongly when an expression contains multiple operators.

In conventional arithmetic expressions, the usual precedence order is:

1. Parentheses have the highest grouping priority because they explicitly determine the expression structure.
2. Multiplication and division have higher precedence than addition and subtraction.
3. Addition and subtraction have lower precedence than multiplication and division.

For example, consider:

a + b * c

The correct interpretation is:

[

a+(b*c)

]

The multiplication operation is grouped first, and its result becomes the right operand of addition.

If addition were incorrectly grouped first, the expression would become:

[

(a+b)*c

]

This is a different expression and can produce a different numerical result.

For example, let (a=2), (b=3), and (c=4).

Correct interpretation:

[

2+(3*4)=2+12=14

]

Incorrect interpretation:

[

(2+3)*4*=*5*4=20

]

The results differ because the expression has been grouped incorrectly.

3. Understanding Operator Associativity

Associativity determines how operators with equal precedence are grouped when multiple instances occur in the same expression.

A. Left Associativity

In left associativity, the expression is grouped from left to right.

For example:

a + b + c

The grouping is:

[

(a+b)+c

]

Similarly:

a * b * c

is conventionally grouped as:

[

(a*b)*c

]

B. Right Associativity

In right associativity, the expression is grouped from right to left.

For example, if exponentiation is right-associative:

a ^ b ^ c

the intended grouping is:

[

a^{(b^c)}

]

Therefore, precedence and associativity perform different functions. Precedence establishes priority between different operators, whereas associativity determines grouping among operators at the same precedence level.

4. Identification of the Key Issue

The key issue in the given scenario is **operator precedence ambiguity caused by an insufficiently structured grammar**.

The original grammar is:

E → E + E

| E * E

| id

Both addition and multiplication are defined at the same grammatical level. As a result, the grammar does not specify that multiplication must be grouped before addition.

For the input:

id + id * id

the grammar permits two distinct parse trees.

Parse Tree 1: Correct Precedence

The intended grouping is:

[

id_1+(id_2*id_3)

]

The corresponding AST is:

```
      +
     / \
   id₁  *
       / \
     id₂ id₃
```

In this tree:

- The root node is addition.
- The left child is the first identifier.
- The right child is multiplication.
- The multiplication node contains the second and third identifiers.

This tree represents the intended arithmetic expression.

Parse Tree 2: Incorrect Precedence

The alternative grouping is:

[

(id_1+id_2)*id_3

]

Its AST is:

```
      *
     / \
    +   id₃
   / \
 id₁ id₂
```

In this tree, addition is performed before multiplication. This interpretation does not follow conventional arithmetic precedence.

Since both structures can be generated by the original grammar, the grammar is ambiguous.

5. How the Parser Encounters a Conflict

A parser using the ambiguous grammar cannot determine the intended grouping from the grammar alone.

In a bottom-up shift-reduce parser, the conflict can occur after the parser has recognized the first two identifiers and the addition operator.

The input is:

id + id * id

After processing the first part of the expression, the parser must decide whether to reduce the addition expression or shift the multiplication operator.

Option 1: Reduce addition first

The parser may group the first two identifiers as an addition expression:

[

id_1+id_2

]

It can then combine that expression with the final identifier using multiplication:

[

(id_1+id_2)*id_3

]

This produces the incorrect precedence structure for conventional arithmetic.

Option 2: Shift multiplication

The parser shifts the multiplication operator and processes the multiplication expression first:

[

id_2*id_3

]

It then combines this result with the first identifier through addition:

[

id_1+(id_2*id_3)

]

This produces the intended structure.

The decision between shifting and reducing is therefore related to operator precedence. A parser using the ambiguous grammar without precedence declarations cannot derive the intended rule from the grammar itself.

6. Grammar Modification Using Precedence Levels

The recommended solution is to rewrite the grammar so that operators with different precedence levels are represented by different non-terminals.

The modified grammar is:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

F → id

Here:

- E represents an expression involving addition.
- T represents a term involving multiplication.
- F represents a factor, such as an identifier.

The production E → E + T defines addition at the expression level, while T → T * F defines multiplication at the term level.

Because multiplication is handled within T and addition is handled within E, multiplication has higher precedence than addition.

To support parenthesized expressions, the factor production can be extended as follows:

F → id | (E)

The complete grammar becomes:

E → E + T | T

T → T * F | F

F → id | (E)

This grammar encodes the intended precedence hierarchy.

7. Derivation of the Correct Expression

Consider the input:

$id_1 + id_2 * id_3$

The expression can be recognized according to the modified grammar as follows:

Step 1: The first identifier is recognized as a factor using F → id.

Step 2: The factor is recognized as a term using T → F.

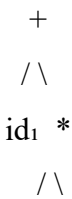
Step 3: The term is recognized as an expression using E → T.

Step 4: The second and third identifiers are recognized as factors.

Step 5: These factors are combined into a multiplication term using T → T * F.

Step 6: The first expression and the multiplication term are combined using E → E + T.

The resulting AST is:



id_2 id_3

Therefore, the expression is interpreted as:

[

id_1+(id_2*id_3)

]

The grammar correctly enforces multiplication precedence.

8. Removing Left Recursion for Predictive Parsing

The modified grammar contains left recursion:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

Left recursion is unsuitable for a conventional recursive-descent predictive parser because it can cause infinite recursive calls.

Therefore, the grammar can be transformed into the following form:

$E \rightarrow T\ E'$

$E' \rightarrow + T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow * F\ T' \mid \varepsilon$

$F \rightarrow id \mid (E)$

Here, E' represents the continuation of an expression, and T' represents the continuation of a term.

The transformed grammar preserves the precedence hierarchy:

- Addition is handled by E and E'.
- Multiplication is handled by T and T'.
- Identifiers and parenthesized expressions are handled by F.

The FIRST and FOLLOW sets must then be calculated to construct the predictive parsing table and verify that the grammar is suitable for LL(1) parsing.

When building the AST, the parser should also preserve left associativity by accumulating repeated operations from left to right.

9. Alternative Solution Using Precedence and Associativity Declarations

Some parser generators allow precedence and associativity declarations to resolve shift-reduce conflicts.

For example, a parser may be configured with the following rules:

- Multiplication has higher precedence than addition.
- Addition is left-associative.
- Multiplication is left-associative.

When the parser encounters the input id + id * id, these declarations tell it to shift or process multiplication before reducing the addition expression.

This approach can be useful when using a bottom-up parser such as an LR-family parser.

However, precedence declarations should be used carefully and verified against the language specification. For an academic grammar-design task, separating the grammar into expression, term, and factor levels is often the clearest solution.

10. Decision and Justification

The syntax analyzer should not resolve operator conflicts through arbitrary production selection. The grammar or parsing technique must explicitly define the intended precedence and associativity rules.

The following decisions should be taken:

1. **Separate precedence levels:** Define addition at the expression level and multiplication at the term level.
2. **Define associativity:** Ensure that repeated addition and multiplication are grouped according to their specified associativity.
3. **Support parentheses:** Use F → id | (E) so that explicit grouping is correctly represented.
4. **Remove left recursion when necessary:** Transform the grammar if implementing a conventional predictive parser.
5. **Recalculate FIRST and FOLLOW sets:** Verify the transformed grammar's suitability for LL(1) parsing.
6. **Configure parser precedence when appropriate:** If using a parser generator, specify precedence and associativity declarations where supported.
7. **Test the resulting AST:** Validate expressions involving several operators and parentheses to ensure that the correct structure is produced.

These decisions ensure that the parser generates a syntactically and semantically appropriate expression structure.

11. Expected Result

For the input:

id + id * id

the expected AST is:

+

/ \

id *

/ \

id id

The tree represents:

[

id_1+(id_2*id_3)

]

Therefore, multiplication is grouped before addition.

For example, if the identifiers represent 2, 3, and 4, respectively, the expression evaluates to:

[

2+(3*4)=2+12=14

]

The alternative grouping would produce:

[

(2+3)*4*=*54=20

]

This demonstrates why the grammar must enforce the intended precedence rules.

12. Conclusion

The operator precedence conflict scenario demonstrates that grammar design is essential for correct arithmetic expression parsing. The original grammar E → E + E | E * E | id is ambiguous because it does not distinguish the precedence of addition and multiplication or explicitly establish associativity. For the expression id + id * id, the intended interpretation is id + (id * id), because multiplication has higher precedence than addition. If the parser constructs the alternative tree (id + id) * id, the resulting evaluation may be incorrect.

The appropriate decision is to rewrite the grammar using separate expression, term, and factor levels or to use a parsing technique with clearly defined precedence and associativity declarations. For predictive parsing, left recursion must be eliminated and FIRST/FOLLOW sets must be checked. These measures enable the syntax analyzer to construct the correct AST, avoid parsing conflicts, and ensure accurate arithmetic evaluation.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand